

Programação Competitiva

Semana 7: Busca binária

Alberto Tavares Duarte Neto

01 de Outubro, 2025

Universidade de Brasília

Problema

Considere um vetor $v_1, \dots, v_n$ **ordenado** ($n \leq 10^5$). Teremos $q \leq 10^5$ consultas. Em cada consulta recebemos um inteiro x , e queremos encontrar o menor índice i tal que $v_i \geq x$.

Problema

Considere um vetor $v_1, \dots, v_n$ **ordenado** ($n \leq 10^5$). Teremos $q \leq 10^5$ consultas. Em cada consulta recebemos um inteiro x , e queremos encontrar o menor índice i tal que $v_i \geq x$.

Considere um índice j . Como o vetor v está ordenado, temos os seguintes casos:

Problema

Considere um vetor $v_1, \dots, v_n$ **ordenado** ($n \leq 10^5$). Teremos $q \leq 10^5$ consultas. Em cada consulta recebemos um inteiro x , e queremos encontrar o menor índice i tal que $v_i \geq x$.

Considere um índice j . Como o vetor v está ordenado, temos os seguintes casos:

- $v_j < x$, e pela ordenação, nenhum índice k **menor** que j pode ser a resposta, pois $v_k \leq v_j < x$.

Problema

Considere um vetor $v_1, \dots, v_n$ **ordenado** ($n \leq 10^5$). Teremos $q \leq 10^5$ consultas. Em cada consulta recebemos um inteiro x , e queremos encontrar o menor índice i tal que $v_i \geq x$.

Considere um índice j . Como o vetor v está ordenado, temos os seguintes casos:

- $v_j < x$, e pela ordenação, nenhum índice k **menor** que j pode ser a resposta, pois $v_k \leq v_j < x$.
- $v_j \geq x$, nenhum índice k **maior** que j pode ser a resposta, pois $x \leq v_j \leq v_k$.

Problema

Considere um vetor $v_1, \dots, v_n$ **ordenado** ($n \leq 10^5$). Teremos $q \leq 10^5$ consultas. Em cada consulta recebemos um inteiro x , e queremos encontrar o menor índice i tal que $v_i \geq x$.

Considere um índice j . Como o vetor v está ordenado, temos os seguintes casos:

- $v_j < x$, e pela ordenação, nenhum índice k **menor** que j pode ser a resposta, pois $v_k \leq v_j < x$.
- $v_j \geq x$, nenhum índice k **maior** que j pode ser a resposta, pois $x \leq v_j \leq v_k$.

Ou seja, ao verificar o índice k , podemos retirar do nosso espaço de busca ou os índices maiores que k , ou os índices menores que k . Como não sabemos qual será o caso, escolhemos um k que divida na metade.

Busca binária

Passo a passo:

Passo a passo:

- Mantemos dois inteiros l e r , correspondendo ao intervalo $[l, r]$ que ainda deve ser verificado.

Busca binária

Passo a passo:

- Mantemos dois inteiros l e r , correspondendo ao intervalo $[l, r]$ que ainda deve ser verificado.
- Escolhemos um índice $mid = (l + r)/2$, que divide o intervalo $[l, r]$ em dois subintervalos com diferença de tamanho no máximo 1.

Busca binária

Passo a passo:

- Mantemos dois inteiros l e r , correspondendo ao intervalo $[l, r]$ que ainda deve ser verificado.
- Escolhemos um índice $mid = (l + r)/2$, que divide o intervalo $[l, r]$ em dois subintervalos com diferença de tamanho no máximo 1.
- Verificamos o elemento v_{mid} para saber se ficamos com o intervalo $[l, mid - 1]$ ou $[mid + 1, r]$.

Busca binária

Passo a passo:

- Mantemos dois inteiros l e r , correspondendo ao intervalo $[l, r]$ que ainda deve ser verificado.
- Escolhemos um índice $mid = (l + r)/2$, que divide o intervalo $[l, r]$ em dois subintervalos com diferença de tamanho no máximo 1.
- Verificamos o elemento v_{mid} para saber se ficamos com o intervalo $[l, mid - 1]$ ou $[mid + 1, r]$.
- Resolvemos agora o novo intervalo.

Busca binária

Passo a passo:

- Mantemos dois inteiros l e r , correspondendo ao intervalo $[l, r]$ que ainda deve ser verificado.
- Escolhemos um índice $mid = (l + r)/2$, que divide o intervalo $[l, r]$ em dois subintervalos com diferença de tamanho no máximo 1.
- Verificamos o elemento v_{mid} para saber se ficamos com o intervalo $[l, mid - 1]$ ou $[mid + 1, r]$.
- Resolvemos agora o novo intervalo.

A função de custo deste algoritmo é dada por

$$T(n) = T(n/2) + O(1)$$

A complexidade final é, portanto,

Busca binária

Passo a passo:

- Mantemos dois inteiros l e r , correspondendo ao intervalo $[l, r]$ que ainda deve ser verificado.
- Escolhemos um índice $mid = (l + r)/2$, que divide o intervalo $[l, r]$ em dois subintervalos com diferença de tamanho no máximo 1.
- Verificamos o elemento v_{mid} para saber se ficamos com o intervalo $[l, mid - 1]$ ou $[mid + 1, r]$.
- Resolvemos agora o novo intervalo.

A função de custo deste algoritmo é dada por

$$T(n) = T(n/2) + O(1)$$

A complexidade final é, portanto, $O(\log n)$.

Código da busca binária

Implementação da busca binária

```
int l = 0, r = n-1, resposta;
while(l <= r) {
    int mid = (l+r)/2;

    if(v[mid] < X) {
        l = mid+1;
    }
    else if(v[mid] > X) {
        r = mid-1;
    }
    else {
        resposta = mid;
        break;
    }
}
```

Lower bound

O c++ possui uma função que recebe os iterators de um vector e um inteiro x , e retorna o primeiro elemento maior ou igual a x de um vetor ordenado: a função *lower_bound*.

A implementação do `lower_bound` é uma busca binária utilizado estes iterators.

Lower bound

O c++ possui uma função que recebe os iterators de um vector e um inteiro x , e retorna o primeiro elemento maior ou igual a x de um vetor ordenado: a função *lower_bound*.

A implementação do *lower_bound* é uma busca binária utilizando estes iterators.

Observação. Não confundir o método *lower_bound* do set com a função *lower_bound*. Utilizar a função *lower_bound* no set te dará TLE!

Lower bound em ação

```
vector<int> v(n);  
// leitura do vector  
// ...  
auto it = lower_bound(v.begin(), v.end(), x);  
cout << it - v.begin() << endl; // indice do primeiro maior  
                                // ou igual x
```


Por que busca binária

Por que não podemos apenas utilizar `lower_bounds` e `sets`?

- Existem problemas interativos: existe um vetor escondido, e você pode fazer poucas consultas neste vetor para determinar alguma informação (problema B da lista).

Por que busca binária

Por que não podemos apenas utilizar `lower_bounds` e `sets`?

- Existem problemas interativos: existe um vetor escondido, e você pode fazer poucas consultas neste vetor para determinar alguma informação (problema B da lista).
- Existem problemas em que o espaço de busca não é um vetor de inteiros, e sim um intervalo de números reais.

Por que busca binária

Por que não podemos apenas utilizar `lower_bounds` e `sets`?

- Existem problemas interativos: existe um vetor escondido, e você pode fazer poucas consultas neste vetor para determinar alguma informação (problema B da lista).
- Existem problemas em que o espaço de busca não é um vetor de inteiros, e sim um intervalo de números reais.
- Uma aplicação da busca binária, chamada de **busca binária na resposta**, resolve problemas em que verificar alguma propriedade em função de um inteiro x é custoso, mas é monotônico (veremos a seguir).

Um problema

Problema

É dado um vetor $v_1, \dots, v_n$ de inteiros, com $n \leq 10^5$, e um inteiro $k \leq n$.

Um problema

Problema

É dado um vetor $v_1, \dots, v_n$ de inteiros, com $n \leq 10^5$, e um inteiro $k \leq n$.

Você deve dividir o vetor em k subvetores contíguos de forma que a maior soma de um subvetor seja **minimizada**.

Um problema

Problema

É dado um vetor $v_1, \dots, v_n$ de inteiros, com $n \leq 10^5$, e um inteiro $k \leq n$.

Você deve dividir o vetor em k subvetores contíguos de forma que a maior soma de um subvetor seja **minimizada**.

Encontrar um algoritmo que determine imediatamente a resposta é difícil. Porém, conseguimos verificar, para um dado x , se a resposta é maior que x ou não.

Um problema

Problema

É dado um vetor $v_1, \dots, v_n$ de inteiros, com $n \leq 10^5$, e um inteiro $k \leq n$.

Você deve dividir o vetor em k subvetores contíguos de forma que a maior soma de um subvetor seja **minimizada**.

Encontrar um algoritmo que determine imediatamente a resposta é difícil. Porém, conseguimos verificar, para um dado x , se a resposta é maior que x ou não.

Ao "chutar" o valor x , conseguimos dividir o vetor gulosamente: adicionamos elementos até que o próximo elemento faça a soma deste subvetor ultrapassar x . Verificamos se o número de subvetores é $\leq k$.

Um problema

Problema

É dado um vetor $v_1, \dots, v_n$ de inteiros, com $n \leq 10^5$, e um inteiro $k \leq n$.

Você deve dividir o vetor em k subvetores contíguos de forma que a maior soma de um subvetor seja **minimizada**.

Encontrar um algoritmo que determine imediatamente a resposta é difícil. Porém, conseguimos verificar, para um dado x , se a resposta é maior que x ou não.

Ao "chutar" o valor x , conseguimos dividir o vetor gulosamente: adicionamos elementos até que o próximo elemento faça a soma deste subvetor ultrapassar x . Verificamos se o número de subvetores é $\leq k$.

Solução: fazer busca binária no conjunto das possíveis resposta. Ao chutar um valor x , verificar se conseguimos uma divisão cuja maior soma seja $\leq x$.

Generalizando

Seja $P : \{1, \dots, m\} \rightarrow \{0, 1\}$ uma função. Definiremos $P(x) = 1$ se uma propriedade com respeito a x vale, e $P(x) = 0$ caso contrário. Por exemplo, no problema anterior definimos que $P(x) = 1$ se conseguimos dividir o vetor em k subvetores cuja maior soma seja $\leq x$, e $P(x) = 0$ caso contrário.

Generalizando

Seja $P : \{1, \dots, m\} \rightarrow \{0, 1\}$ uma função. Definiremos $P(x) = 1$ se uma propriedade com respeito a x vale, e $P(x) = 0$ caso contrário. Por exemplo, no problema anterior definimos que $P(x) = 1$ se conseguimos dividir o vetor em k subvetores cuja maior soma seja $\leq x$, e $P(x) = 0$ caso contrário.

Se a função P for **não decrescente**, podemos realizar uma busca binária para encontrar o menor inteiro x tal que $P(x) = 1$.

Generalizando

Seja $P : \{1, \dots, m\} \rightarrow \{0, 1\}$ uma função. Definiremos $P(x) = 1$ se uma propriedade com respeito a x vale, e $P(x) = 0$ caso contrário. Por exemplo, no problema anterior definimos que $P(x) = 1$ se conseguimos dividir o vetor em k subvetores cuja maior soma seja $\leq x$, e $P(x) = 0$ caso contrário.

Se a função P for **não decrescente**, podemos realizar uma busca binária para encontrar o menor inteiro x tal que $P(x) = 1$.

- Em certo passo, nosso espaço de busca é $[l, r]$.

Generalizando

Seja $P : \{1, \dots, m\} \rightarrow \{0, 1\}$ uma função. Definiremos $P(x) = 1$ se uma propriedade com respeito a x vale, e $P(x) = 0$ caso contrário. Por exemplo, no problema anterior definimos que $P(x) = 1$ se conseguimos dividir o vetor em k subvetores cuja maior soma seja $\leq x$, e $P(x) = 0$ caso contrário.

Se a função P for **não decrescente**, podemos realizar uma busca binária para encontrar o menor inteiro x tal que $P(x) = 1$.

- Em certo passo, nosso espaço de busca é $[l, r]$.
- Chute $mid = (l + r)/2$ e calcule $P(mid)$.

Generalizando

Seja $P : \{1, \dots, m\} \rightarrow \{0, 1\}$ uma função. Definiremos $P(x) = 1$ se uma propriedade com respeito a x vale, e $P(x) = 0$ caso contrário. Por exemplo, no problema anterior definimos que $P(x) = 1$ se conseguimos dividir o vetor em k subvetores cuja maior soma seja $\leq x$, e $P(x) = 0$ caso contrário.

Se a função P for **não decrescente**, podemos realizar uma busca binária para encontrar o menor inteiro x tal que $P(x) = 1$.

- Em certo passo, nosso espaço de busca é $[l, r]$.
- Chute $mid = (l + r)/2$ e calcule $P(mid)$.
- Se $P(mid) = 1$, então restringimos nosso espaço de busca para $[l, mid - 1]$.

Generalizando

Seja $P : \{1, \dots, m\} \rightarrow \{0, 1\}$ uma função. Definiremos $P(x) = 1$ se uma propriedade com respeito a x vale, e $P(x) = 0$ caso contrário. Por exemplo, no problema anterior definimos que $P(x) = 1$ se conseguimos dividir o vetor em k subvetores cuja maior soma seja $\leq x$, e $P(x) = 0$ caso contrário.

Se a função P for **não decrescente**, podemos realizar uma busca binária para encontrar o menor inteiro x tal que $P(x) = 1$.

- Em certo passo, nosso espaço de busca é $[l, r]$.
- Chute $mid = (l + r)/2$ e calcule $P(mid)$.
- Se $P(mid) = 1$, então restringimos nosso espaço de busca para $[l, mid - 1]$.
- Se $P(mid) = 0$, então restringimos nosso espaço de busca para $[mid + 1, r]$.

Suponhamos que em cada passo da busca binária seja realizado uma função em $O(n)$ (como iterar um vetor de tamanho, por exemplo).

A complexidade pode ser definida como

$$T(n) = T(n/2) + O(n) .$$

Logo, a complexidade final é $T(n) = O(n \log n)$.